

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier

Zeitplan bis zum Jahresende

08.01.2026	Datenflussanalyse	Übungsblatt „Datenflussanalyse“
13.01.2026	Datenflussanalyse von TRIPLA	
15.01.2026	Projekt IV (Kontrollflussanalyse)	Abgabe Übungsblatt „Datenflussanalyse“
20.01.2026	Code Clone Detection	
22.01.2026	Reverse Engineering, Erkennung von Entwurfsmustern	Abgabe Projekt IV
27.01.2026	Gastvortrag: „Programm Analysis and Transformation from a Practitioner’s Point of View“ (Jonas Eckstein, itestra)	Projekt V (Datenflussanalyse)
29.01.2026	Programmtransformationen	Übungsblatt „Programmtransformationen“
03.02.2026	Partielle Evaluation von TRIPLA	
05.02.2026	Higher-Order TRIPLA	
10.02.2026	Details zum Portfolio + Tipps zu Projekt V	Abgabe + Übung ÜTRAFO
17.03.2026	Abgabe Portfolio inkl. Projekt V	

Semantik- erhaltende Programm- transformationen

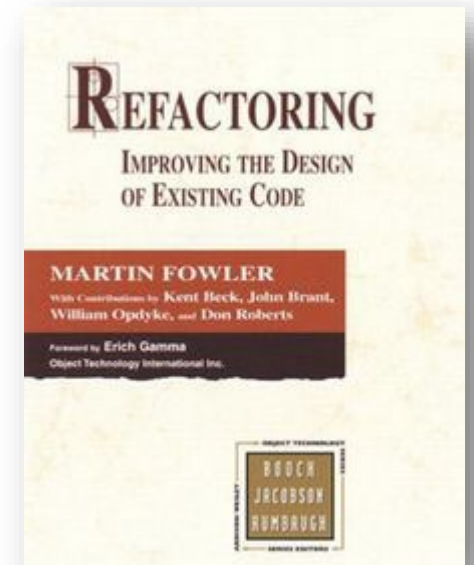


Bildquelle: mit KI erzeugt (Bing, DALL-E)

Programmtransformationen

- **Refactorings** → In der Praxis erprobte semantik-erhaltende Transformationen samt Beschreibung (wann, warum, wie) und Beispielen. (Best Practice)

- Rename method
- Extract method
- ...



- Nicht jede semantik-erhaltende Transformation ist ein Refactoring!

Extract Method

```
void printExamResult() {  
    printStudentDetails();  
  
    System.out.println("Module: " + module.getName());  
    System.out.println("Grade: " + module.getGrade());  
}
```



```
void printExamResult() {  
    printStudentDetails();  
    printModuleResult();  
}  
  
void printModuleResult() {  
    System.out.println("Module: " + module.getName());  
    System.out.println("Grade: " + module.getGrade());  
}
```

Typische Programmtransformationen in Compilern

- Dead code elimination (siehe DFA)
- Common subexpression elimination (siehe DFA)
- Constant folding ($50 \cdot 3 + 7 \rightarrow 157$)

Auf den nächsten Folien

- Constant Propagation
- Strength Reduction
- Desugaring
- Inlining
- Loop optimizations

Constant Propagation

```
f(x,y)
{ z=5;
  while (x>0)
  { x = x-z;
  }
}
```

```
f(x,y)
{ z=5;
  while (x>0)
  { x = x-5;
  }
}
```

Strength Reduction

- Ersetzen durch „billigere Operation“: $a = 2 * b \rightarrow a = b + b$

```
i=0;  
c=42;  
while (i<a.length) { a[i]=i*c; i=i+1;}
```

```
i=0;  
c=42;  
v=0;  
while (i<a.length)  
{ a[i]=v;  
  v=v+c;  
  i=i+1;  
}
```



Desugaring

```
for(int i=0; i<100; i=i+1) { s=s+i; }
```

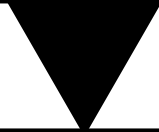
```
int i=0; while (i<100) { s=s+i; i=i+1; }
```

```
List<Integer> list = List.of(1, 3, 6, 8, 9);  
for (int i : list) { s = s+i; }
```

```
List<Integer> list = List.of(1, 3, 6, 8, 9);  
Iterator<Integer> itr = list.iterator();  
while(itr.hasNext()) { s = s+itr.next(); }
```

Inlining

```
max(x,y) { if (x>y) return x; else return y; }  
  
i=0;  
m=0;  
while (i<a.length) { m = max(m,a[i]); i=i+1; }
```



```
i=0;  
m=0;  
while (i<a.length)  
{ if (m>a[i]) r=m; else r=a[i];  
  m = r;  
  i=i+1;  
}
```

Loop Unrolling

```
for(int i=0; i<5; i++)  
    print(i);
```



```
print(0);  
print(1);  
print(2);  
print(3);  
print(4);  
print(5);
```

```
for(int i=0; i<4*k; i++)  
    print(i);
```



```
for(int i=0; i<k; i++)  
    print(i);  
    print(i+1);  
    print(i+2);  
    print(i+3);
```

- Reduziert die Anzahl der Schleifentests, die ausgeführt werden müssen.

Loop Unrolling

- Ersten und/oder letzten Schleifendurchlauf herausziehen, um ihn zu „optimieren“

```
int factorial(int n) {  
    int fact = 1;  
    for(int i=1; i<n+1; i++)  
        fact *= i;  
    return(fact);  
}
```

```
int factorial(int n) {  
    int fact = 1;  
    if (1<n+1) fact *= 1;  
    for(int i=2; i<n+1; i++)  
        fact *= i;  
    return(fact);  
}
```

```
int factorial(int n) {  
    int fact = 1;  
  
    for(int i=2; i<n+1; i++)  
        fact *= i;  
    return(fact);  
}
```

Loop Fusion/Fission

```
for(int i=0; i<n; i++) {  
    sum += array[i];  
    product *= array[i];  
}
```

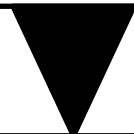
Loop Fission

```
for(int i=0; i<n; i++) {  
    sum += array[i];  
}  
  
for(int i=0; i<n; i++) {  
    product *= array[i];  
}
```

Loop Fusion
(aka Loop Jamming)

Loop Inversion (aka Loop Reversal)

```
for (int i=0; i<10; i++)  
    result += f(i);
```



```
for (int i=0; i<10; i++)  
    result += f(9-i);
```

Loop-Invariant Code Motion

```
for (int i=0; i<10; i++)  
    result = sin(x)/cos(x)+i;
```



```
t = sin(x)/cos(x);  
for (int i=0; i<10; i++)  
    result = t + i;
```

Loop Interchange

```
for (int i=0; i<10; i++)  
    for (int j=0; j<10; j++)  
        result += a[i][j];
```



```
for (int j=0; j<10; j++)  
    for (int i=0; i<10; i++)  
        result += a[i][j];
```

Weitere Programmtransformationen auf den nächsten Folien

- Obfuscation
- Pass Separation
- Partial Evaluation

```
private void
CalculatePayroll(SpecialList
employeeGroup) {
    while (employeeGroup.HasMore()) {
        employee =
employeeGroup.GetNext(true);
        employee.UpdateSalary();
        DistributeCheck(employee);
    }
}
```

```
private void a(a b) {
    while (b.a()) {
        a = b.a(true);
        a.a();
        a(a);
    }
}
```

Original Source Code Before String Encryption

Reverse-Engineered Source Code After String Encryption

```
...
MessageBox.show("Invalid
Authentication - Try Again")
...
```

```
...
MessageBox.show(a.b("¥βΣημ.πst☺£∞_jfi_●ωff"))
...
```

Original Source Code Before Control Flow Obfuscation

Reverse-Engineered Source Code After Control Flow Obfuscation

```
public int CompareTo(Object o) {
    int n = occurrences -
        ((WordOccurrence) o).occurrences;
    if (n == 0) {
        n = String.Compare
            (word, ((WordOccurrence) o).word);
    }
    return n;
}
```

```
public virtual int _a(Object A_0) {  
    int local0;  
    int local1;  
    local 10 = this.a - (c) A_0.a;  
    if (local0 != 0) goto i0;  
    while (true) {  
        return local1;  
    }  
    i1: local10 =  
System.String.Compare(this.b, (c)  
A_0.b);  
    goto i0;  
}
```

Pass Separation

```
def sqsum(values):  
    sum=0;  
    for v in values:  
        sum=sum+v*v  
    return sum  
  
print(sqsum([1,2,3]))
```

```
def sqsum(values):  
    sum=0;  
    sqvalues = [ v*v for v in values]  
    for v in sqvalues:  
        sum=sum+v  
    return sum  
  
print(sqsum([1,2,3]))
```

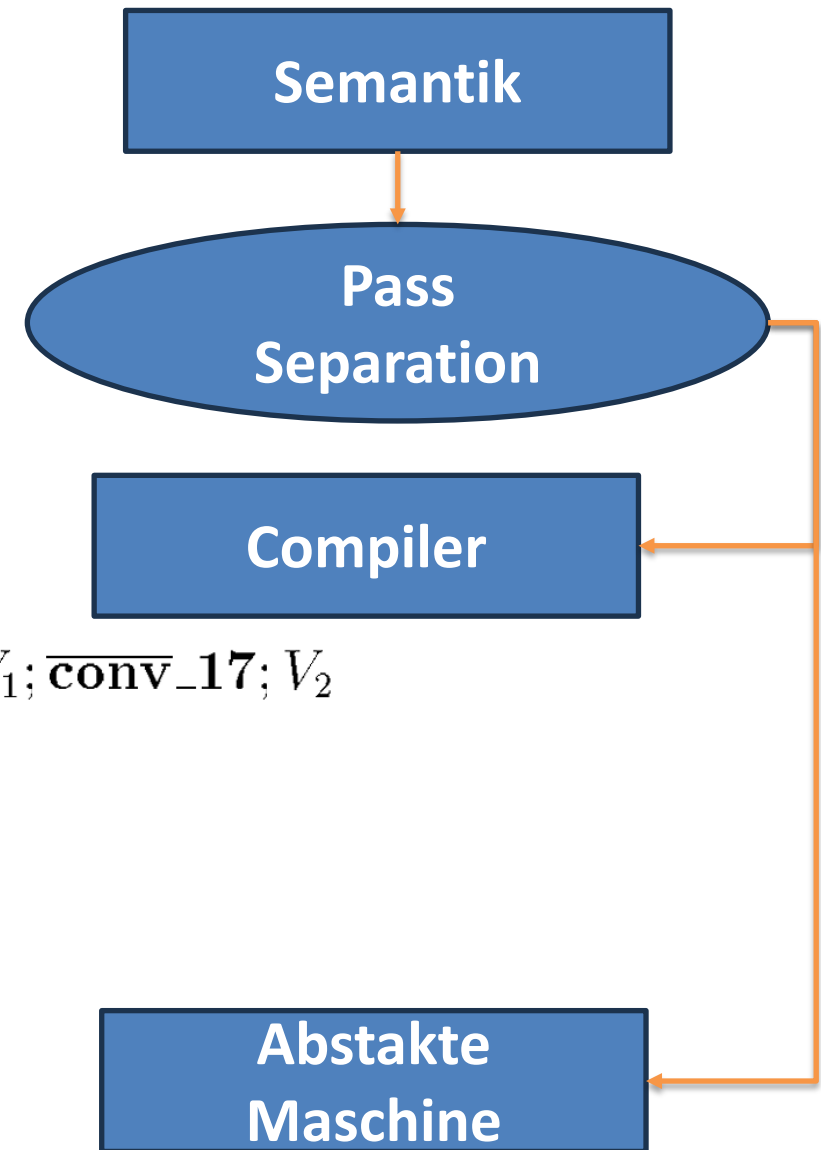

Recursive Functions

$$\begin{array}{c}
 \text{newind} \triangleright R \rightarrow N \\
 V_1 \triangleright [[\text{red}(N, \text{ind}(N))|R], \text{replace}(X, \text{ind}(N), E)] \rightarrow [R', V_1'] \\
 V_2 \triangleright [\text{replace_red}(N, V_1', R'), \text{replace}(X, \text{val}(V_1'), E)] \rightarrow [R'', V_2'] \\
 \hline
 \text{letrec } X = V_1 \text{ in } V_2 \text{ end} \triangleright [R, E] \rightarrow [R'', V_2']
 \end{array}$$

$$\text{letrec } X = V_1 \text{ in } V_2 \text{ end} \quad \Rightarrow \quad \overline{\text{letrec}}; \overline{\text{newind}}; \overline{\text{conv_16}}(X); V_1; \overline{\text{conv_17}}; V_2$$

Recursive Functions

$$\begin{array}{l}
 \langle \overline{\text{letrec}}; C, [D, [R, E]] \rangle \Rightarrow \langle C, [[[R, E]|D], R] \rangle \\
 \langle \overline{\text{newind}}; C, [D, R] \rangle \Rightarrow \langle C, [D, \text{new_index}(R)] \rangle \\
 \langle \overline{\text{conv_16}}(X); C, [[[R, E]|D], N] \rangle \Rightarrow \\
 \quad \langle C, [[[N, X, E]|D], [[\text{red}(N, \text{ind}(N))|R], \text{replace}(X, \text{ind}(N), E)]] \rangle \\
 \langle \overline{\text{conv_17}}; C, [[[N, X, E]|D], [R, V]] \rangle \Rightarrow \\
 \quad \langle C, [D, [\text{replace_red}(N, V, R), \text{replace}(X, \text{val}(V), E)]] \rangle
 \end{array}$$



Partial Evaluation

```
pow(n, x) = if (n==0) then 1 else x*pow(n-1, x)
```

```
pow(3, x)  -> if (3==0) then 1 else x*pow(3-1, x)
            -> x*pow(2,x)
            -> x*(if (2==0) then 1 else x*pow(2-1, x))
            -> x*x*pow(1,x)
            -> x*x*(if (1==0) then 1 else x*pow(1-1, x))
            -> x*x*x*pow(0,x)
            -> x*x*x*(if (0==0) then 1 else x*pow(0-1, x))
            -> x*x*x*1
            -> x*x*x
```

```
pow(n, 3)  -> ???
```

Partial Evaluation

```
while(x>0) { x=x-1;  
            while(x>0) { x=x-1;  
                        y=y-1;  
                        }  
            }  
return y+42;
```

Annahme: x=5

```
y=y-1;  
y=y-1;  
y=y-1;  
y=y-1;  
return y+42;
```

Partial Evaluation

```
res=1
while(n!=0) { res=res*x;
              n=n-1 }
return res;
```

Annahme: n=3, x unbekannt

```
res=x;
res=res*x;
res=res*x;
return res;
```

Annahme: n unbekannt, x = 2

```
res=1
while(n!=0) { res=res*2;
              n=n-1 }
return res;
```

```
res=1
while(n!=0) { res=res*x;
              x=2*x
              n=n-1 }
return res;
```

Annahme: n=3, x unbekannt

```
res=x;
x=2*x
res=res*x;
x=2*x
res=res*x;
x=2*x
return res;
```

Annahme: n unbekannt, x = 2

keine Auswertung möglich,
da sich der Wert von x
bei jeder Iteration ändert

Partial Evaluation

```
res=1
while(n!=0) { res=res*x;
              n=n-1 }
return res;
```

Annahme: n=3, x unbekannt

```
res=x;
res=res*x;
res=res*x;
return res;
```

Annahme: n unbekannt, x = 2

```
res=1
while(n!=0) { res=res*2;
              n=n-1 }
return res;
```

```
res=1
while(n!=0) { res=res*x;
              x=n*x
              n=n-1 }
return res;
```

Annahme: n=3, x unbekannt

```
res=x;
x=3*x
res=res*x;
x=2*x
res=res*x;
x=1*x
return res;
```

Annahme: n unbekannt, x = 2

keine Auswertung möglich,
da der Wert von x von
unbekanntem Wert der
Variable n abhängt.

Polyvariant Partial Evaluation

```

let f(x,y) { while(x>0) { x=x-1;
                    while(x>0) { x=x-1;
                                y=y-1; } }
    return y+42; }
in
  f(5,z);
  f(2,z+5)

```

```

let f(x,y) { while(x>0) { x=x-1; while(x>0) { x=x-1; y=y-1; } }
    return y+42; }
  f5(y) { y=y-4; return y+42; }
  f2(y) { y=y-1; return y+42; }
in
  f5(z)
  f2(z+5)

```

Futamura Projections

- Interpreter

`python("notify.pl", "in.txt") → o`

- First Futamura projection

`peval(python, "notify.pl") → g`

where $g(\text{"in.txt"}) = o$

g is compiled version of `notify.pl`

- Second Futamura projection

`peval(peval, python) → c`

where $c(\text{"notify.pl"}) = g$

c is a python compiler

- Third Futamura projection

`peval(peval, peval) → z`

where $z(\text{python}) = c$

z is a compiler generator!

`python("notify.pl", "in.txt") → o`

`peval(python, "notify.pl") („in.txt“) → o`

`peval(peval, python) („notify.pl“)(„in.txt“) → o`

`peval(peval, peval) (python)(„notify.pl“)(„in.txt“) → o`